

TRƯỜNG
KHOA

BÀI KIỂM TRA THỰC HÀNH
Học kỳ — Năm học 20.....–20.....

LẬP TRÌNH PHÂN TÍCH DỮ LIỆU

Đề số 02 · Phạm vi Buổi 1–5 · Bộ dữ liệu: Dịch vụ xe công cộng EcoRide

THỜI GIAN LÀM BÀI 60 phút (không kể thời gian phát đề)	TỔNG ĐIỂM 10 điểm — 5 phần, 11 câu	HÌNH THỨC Thực hành trên máy · Jupyter Notebook hoặc Google Colab
THƯ VIỆN ĐƯỢC DÙNG Python chuẩn (re, collections, sqlite3), numpy, pandas, matplotlib, seaborn, scipy, scikit-learn	TÀI LIỆU Được dùng tài liệu cá nhân	KHÔNG ĐƯỢC Dùng Internet, công cụ AI, trao đổi bài

Họ và tên: MSSV: Lớp:

Hướng dẫn làm bài & nộp bài

- Làm bài trong notebook `bai_lam/bai_lam_mau.ipynb`, đổi tên thành `MSSV_HoTen.ipynb`. Mỗi câu làm trong đúng ô được đánh dấu.
- Kết quả phải được in ra trong notebook (bằng `print` hoặc hiển thị bảng). Câu hỏi yêu cầu trả lời bằng lời thì viết vào ô Markdown “Trả lời”.
- Trước khi nộp: chọn **Restart & Run All**. Ô báo lỗi hoặc chưa chạy sẽ không được chấm.
- Nộp 3 tập tin: notebook bài làm, `dong_loi.txt` (câu A1) và `bieu_do.png` (Phần E).

Phân bổ thời gian gợi ý: Phần A 10 phút · Phần B 10 phút · Phần C 18 phút · Phần D 10 phút · Phần E 12 phút.

Buổi	Nội dung được kiểm tra	Câu
1	Python cơ bản · khối lệnh & cấu trúc điều khiển · hàm tự định nghĩa	A1, A2
2	list, tuple, dict, set · mảng & ma trận NumPy · đại số ma trận	A2, B1, B2
3	Tập tin văn bản · tập tin dữ liệu với Pandas · truy cập CSDL · lập trình hướng đối tượng	A1, B3, C1
4	Tiền xử lý: làm sạch dữ liệu · tích hợp dữ liệu · EDA	C2, C3, E
5	Tiền xử lý: biến đổi dữ liệu · thu gọn dữ liệu	D1, D2

Bối cảnh & dữ liệu

EcoRide cho thuê xe đạp, xe đạp điện và xe máy điện qua ứng dụng tại 6 trạm ở Đà Nẵng và Hội An. Khách mở khoá xe ở một trạm, đi rồi trả xe ở một trạm bất kỳ trong cùng thành phố; cước tính theo phút. Bạn nhận dữ liệu từ tháng 3 đến tháng 8/2025 trong thư mục `du_lieu/` gồm 4 tập tin. Dữ liệu là dữ liệu thô, có lỗi — đó là một phần của bài.

Tập tin	Nội dung
nhat_ky_khoa_xe.txt	Nhật ký khoá thông minh của xe điện tháng 7/2025. Mỗi lần mở khoá (MUON) hoặc khoá lại (TRA) ghi một dòng kèm mức pin lúc đó.
chuyen_di.csv	Mỗi dòng là một chuyến đi. Các cột mô tả ở bảng dưới.
khach_hang.xlsx	Sheet khách_hang : ma_kh, nam_sinh, so_dien_thoai, goi_cuoc, ngay_dang_ky Sheet goi_cuoc : goi_cuoc, giam_gia (tỉ lệ, vd. 0.20 = giảm 20%)
he_thong.db	Cơ sở dữ liệu SQLite, 2 bảng. tram : id, ten_tram, thanh_pho, suc_chua (số chỗ đỗ). loai_xe : ma_loai, ten_loai, phi_mo_khoa, gia_phut, gia_30_phut (VNĐ)

Cột trong chuyen_di.csv	Ý nghĩa
ma_cd	Mã chuyến đi, dạng CD + 5 chữ số
bat_dau	Thời điểm mở khoá xe, định dạng ngày/tháng/năm giờ:phút (dd/mm/yyyy HH:MM)
ket_thuc	Thời điểm trả xe, cùng định dạng; có thể trống
tram_muon · tram_tra	Mã trạm mượn xe · mã trạm trả xe
ma_kh	Mã khách có đăng ký gói cước; để trống nếu là khách lẻ (quét mã, trả tiền từng chuyến)
loai_xe	Xe đạp, Xe đạp điện, Xe máy điện
quang_duong_km	Quãng đường GPS ghi nhận được (km)
danh_gia	Rất không hài lòng, Không hài lòng, Bình thường, Hài lòng, Rất hài lòng; có thể trống

Phần A — Python thuần: tập tin văn bản, regex, hàm

2,0 điểm

Phần này **không dùng pandas và numpy**. Chỉ dùng Python chuẩn: `open`, `re`, `collections`...

Một dòng hợp lệ trong `nhat_ky_khoa_xe.txt` có đúng dạng:

```
2025-07-02 07:15 | DN03 | XD014 | TRA | pin=18%
```

- Ngày dạng YYYY-MM-DD và giờ dạng HH:MM, cách nhau đúng một khoảng trắng.
- Mã trạm là DN (Đà Nẵng) hoặc HA (Hội An) + đúng 2 chữ số; mã xe là XD (xe đạp điện) hoặc XM (xe máy điện) + đúng 3 chữ số.
- Sự kiện là một trong hai giá trị viết hoa: MUON (mở khoá), TRA (khóa lại).
- Trường cuối dạng `pin=N%` với N gồm 1–3 chữ số; dòng **kết thúc ngay sau** dấu %.
- Các trường ngăn cách bởi `|` (khoảng trắng – gạch đứng – khoảng trắng).
- Dòng trống và dòng bắt đầu bằng # được bỏ qua — không tính là hợp lệ, cũng không tính là lỗi.

Câu A1. Lọc dòng hợp lệ bằng biểu thức chính quy

1,0 điểm

Đọc tập tin, dùng module `re` phân loại từng dòng thành hợp lệ hoặc không hợp lệ.

- In số dòng hợp lệ và số dòng không hợp lệ.
- Dùng `Counter`, in số sự kiện của từng trạm trong các dòng hợp lệ, sắp xếp theo mã trạm.
- Ghi toàn bộ dòng không hợp lệ (giữ nguyên nội dung, mỗi dòng một dòng) vào tập tin `dong_loi.txt`.

Câu A2. Thống kê pin yếu

1,0 điểm

Viết hàm `ti_le_pin_yeu(cac_dong, nguong=20, toi_thieu=8)` nhận danh sách dòng hợp lệ và trả về dict `{mã xe: tỉ lệ pin yếu}`. **Chỉ xét sự kiện TRA**. Một lần trả xe bị tính là **pin yếu** khi mức pin **nhỏ hơn hẳn** `nguong` (trả xe với `pin=20%` không tính là pin yếu). Tỉ lệ = số lần trả pin yếu ÷ tổng số lần trả của xe đó. Chỉ đưa vào kết quả những xe có ít nhất `toi_thieu` lần trả.

- Gọi hàm với tham số mặc định, in tỉ lệ của từng xe (3 chữ số thập phân) và cho biết xe có tỉ lệ pin yếu cao nhất.
- Dùng kiểu `set`, liệt kê các xe đã có sự kiện ở **cả hai thành phố** (xét tất cả dòng hợp lệ; thành phố là 2 ký tự đầu của mã trạm).

Phần B — NumPy & lập trình hướng đối tượng

2,0 điểm

Câu B1. Ma trận lượt thuê

0,5 điểm

Ma trận `L` (đã có sẵn trong notebook) là số lượt thuê xe trong một ngày thứ Bảy tại 6 trạm DN01, DN02, DN03, DN04, HA01, HA02 (dòng) theo 5 khung giờ 05–08h, 08–11h, 11–14h, 14–17h, 17–20h (cột).

Không dùng vòng lặp.

```
L = np.array([[42, 35, 28, 39, 71],
              [58, 31, 22, 34, 66],
              [25, 47, 44, 41, 52],
              [18, 26, 19, 23, 37],
              [12, 38, 41, 45, 83],
              [33, 29, 24, 36, 48]])
```

- Tổng lượt thuê của từng trạm và trạm cao nhất; trung bình theo từng khung giờ và khung giờ có trung bình cao nhất.
- Tỉ trọng lượt thuê **giờ cao điểm** (khung 05–08h và khung 17–20h) trong tổng lượt của từng trạm — dùng broadcasting; dùng mặt nạ boolean in ra các trạm có tỉ trọng này lớn hơn 0,5.
- Số ô có lượt thuê lớn hơn lượt thuê trung bình **của chính khung giờ đó** (trung bình trên 6 trạm).

Câu B2. Giải hệ phương trình

0,5 điểm

Cước một chuyến xe máy điện được tính theo công thức: cước = phí mở khoá + giá mỗi phút × số phút + phụ phí mỗi km × số km. Ba tham số giá chưa biết; cước của ba chuyến như sau:

Chuyến	Số phút	Số km	Cước (nghìn đồng)
1	20	3	23
2	35	8	42
3	50	9	53

Lập ma trận hệ số và vector vế phải, dùng `np.linalg.solve` tìm ba tham số giá, kiểm tra lại nghiệm bằng phép nhân ma trận, rồi dùng nghiệm dự đoán cước của một chuyến 30 phút, 6 km.

Câu B3. Lớp chuyển đi

1,0 điểm

- Lớp `ChuyenDi(ma, phut, km, gia_phut)` có **thuộc tính lớp** `PHI_MO_KHOA = 5000`. `phut` là **property**; setter ném `ValueError` nếu thời lượng ≤ 0 (áp dụng cả lúc khởi tạo).

- Phương thức `cuoc_phi()` = `PHI_MO_KHOA` + `phut` × `gia_phut`; `toc_do_tb()` = `km` ÷ (`phut` ÷ 60), đơn vị km/h; `__str__` trả về chuỗi đúng mẫu `CD001: 25 phút · 6.0 km · 14.4 km/h · cước 17,500đ`.
- Lớp con `ChuyenDiGoiThang`(`ma`, `phut`, `km`, `gia_phut`, `phut_mien_phi=30`) kế thừa `ChuyenDi`, gọi `super().__init__`, ghi đè `cuoc_phi()`: không thu phí mở khoá, chỉ tính tiền các phút vượt quá `phut_mien_phi` (không bao giờ âm).

Chạy kiểm thử: `in ChuyenDi('CD001', 25, 6.0, 500)`; `in ChuyenDiGoiThang('CD002', 48, 10.4, 500)`; gán `phut` của chuyến `CD001` thành 0 trong khối `try/except` và in thông báo lỗi.

Phần C — Pandas: đọc, làm sạch và tích hợp dữ liệu

2,5 điểm

Câu C1. Đọc dữ liệu từ ba loại nguồn

0,5 điểm

- Đọc `chuyen_di.csv`; chuyển **cả hai** cột `bat_dau`, `ket_thuc` sang kiểu `datetime`, chú ý định dạng ngày/tháng/năm.
- Đọc cả hai sheet của `khach_hang.xlsx`.
- Kết nối `he_thong.db` bằng `sqlite3`, đọc hai bảng bằng `pd.read_sql` với câu lệnh SQL.
- In kích thước (shape) của 5 bảng; in số giá trị thiếu của các cột có giá trị thiếu trong bảng chuyến đi.

Câu C2. Làm sạch bảng chuyến đi

1,0 điểm

Thực hiện **đúng thứ tự** năm bước sau:

1. Loại bỏ các dòng trùng lặp hoàn toàn.
2. Chuẩn hoá `loai_xe` về đúng 3 giá trị `Xe đạp`, `Xe đạp điện`, `Xe máy điện`. Lưu ý dữ liệu có cả cách viết không dấu, và “xe đạp” là phần đầu của “xe đạp điện”.
3. Tạo cột `phut` = số phút từ `bat_dau` đến `ket_thuc`. Đổi các giá trị `phut` ≤ 0 (lỗi đồng hồ) thành giá trị thiếu. Loại bỏ các chuyến có `phut` nhỏ hơn 2 (mở khoá thử) — các chuyến đang trống `phut` phải được giữ lại.
4. Đổi các giá trị `quang_duong_km` âm hoặc lớn hơn 80 (lỗi GPS) thành giá trị thiếu.
5. Tìm ngoại lai `phut` theo quy tắc IQR, với Q1, Q3 **tính riêng cho từng loại xe** (ngoại lai khi > Q3 + 1,5 × IQR). In số ngoại lai của mỗi loại xe, kéo các giá trị đó về đúng ngưỡng trên của loại xe (winsorization). Cuối cùng điền `phut` và `quang_duong_km` còn trống bằng **trung vị theo loại xe**.

In ra: số dòng sau bước 1 và sau bước 3; số giá trị bị đổi thành thiếu ở bước 3 và ở bước 4; số ngoại lai theo loại xe; số chuyến theo từng loại xe sau khi làm sạch.

Câu C3. Tích hợp dữ liệu

1,0 điểm

1. Bảng `loai_xe`: viết code chứng minh `gia_30_phut` là cột dẫn xuất (= `phi_mo_khoa` + 30 × `gia_phut` ở mọi dòng) rồi xoá cột này.
2. Bảng khách hàng: in số mã khách bị trùng; loại trùng sao cho **mỗi ma_kh chỉ còn một dòng**. Chuẩn hoá `so_dien_thoai`: chỉ giữ chữ số, đầu số 84 đổi thành 0 (vd. `+84 905 123 456` → `0905123456`); in số khách có số điện thoại không đủ 10 chữ số. Ghép sheet `goi_cuoc` để mỗi khách có `giam_gia`.

3. Loại các chuyến có `tram_muon` không có trong bảng `tram` (in số dòng bị loại). Ghép chuyến đi với `loai_xe` (khóa `loai_xe` ↔ `ten_loai`), khách hàng và `tram` (khóa `tram_muon` ↔ `id`) sao cho **không mất chuyến nào** còn lại; chuyến không có khách khớp: `goi_cuoc` = "Khách lẻ", `giam_gia` = 0. Gọi bảng kết quả là `df`.

4. Tạo `cuoc` = `phi_mo_khoa` + `phut` × `gia_phut` và `doanh_thu` = `cuoc` × (1 - `giam_gia`).

In ra: số dòng của `df`; số chuyến “Khách lẻ”; tổng doanh thu (có dấu phân cách hàng nghìn); doanh thu theo thành phố và theo tên trạm (triệu đồng, 1 chữ số thập phân, giảm dần).

Phần D — Biến đổi & thu gọn dữ liệu

1,5 điểm

Dùng bảng `df` đã tạo ở câu C3.

Câu D1. Biến đổi dữ liệu

0,75 điểm

- Mã hoá thứ bậc `danh_gia` thành cột `diem_dg`: Rất không hài lòng 1 · Không hài lòng 2 · Bình thường 3 · Hài lòng 4 · Rất hài lòng 5; điền ô trống bằng mode, kiểu số nguyên.
- Tạo cột `log_phut` = $\log(1 + \text{phut})$; in độ lệch (skewness, `scipy.stats.skew`) của `phut` trước và sau biến đổi.
- Rời rạc hoá giờ mở khoá thành cột `khung_gio` bằng `pd.cut`: Sáng [0h, 9h) · Trưa chiều [9h, 16h) · Tan tầm [16h, 20h) · Tối [20h, 24h) — mút trái thuộc khung, mút phải không. In số chuyến mỗi khung; mã hoá one-hot `khung_gio` với tiền tố `kg`, in tên các cột mới.

Câu D2. Thu gọn dữ liệu theo khách hàng

0,75 điểm

- Chỉ xét khách có gói cước (bỏ “Khách lẻ”). Tạo bảng `khach`, mỗi khách một dòng, gồm: `so_chuyen` (số chuyến), `tong_chi` (tổng doanh thu, **ngàn đồng**), `phut_tb`, `km_tb`, `diem_tb` (trung bình thời lượng, quãng đường, điểm đánh giá), `ti_le_xe_may` (tỉ lệ chuyến đi bằng Xe máy điện). In kích thước bảng.
- Chia `tong_chi` thành 4 nhóm **cùng số lượng** Thấp / Trung bình / Khá / Cao; in số khách mỗi nhóm.
- Tính ma trận tương quan 6 cột số; chỉ ra các cặp có $|r| > 0,7$ và giải thích vì sao có thể coi một cột trong mỗi cặp là dư thừa.
- Chuẩn hoá z-score 6 cột số, chạy PCA; in số thành phần ít nhất cần để giữ $\geq 90\%$ phương sai.

Phần E — Trực quan hoá

2,0 điểm

Vẽ **một hình gồm 4 biểu đồ** (2 hàng × 2 cột):

- Trên trái: biểu đồ đường doanh thu theo tháng của **từng loại xe** (3 đường), có điểm đánh dấu và chú thích.
- Trên phải: biểu đồ phân tán `quang_duong_km` theo `phut`, tô màu theo loại xe.
- Dưới trái: biểu đồ **cột ghép** (không chồng) số chuyến theo `khung_gio`, mỗi khung giờ có một cột cho từng thành phố; các khung giờ xếp đúng thứ tự Sáng → Tối.
- Dưới phải: heatmap ma trận tương quan của câu D2, có ghi số trên từng ô, thang màu từ -1 đến 1.

Mỗi biểu đồ có tiêu đề và nhãn cho các trục, ghi rõ đơn vị. Lưu hình thành `bieu_do.png` với độ phân giải 150 dpi. Viết 2 câu nhận xét: (1) xu hướng doanh thu của Xe máy điện so với hai loại xe còn lại; (2) biểu đồ phân tán cho thấy gì về tốc độ di chuyển của ba loại xe.

— HẾT —

Cán bộ coi thi không giải thích gì thêm.